



LMS Node.js API

▼ /signup POST

Parameters:

body:

```
{  
  "name": string,  
  "email": string,  
  "mobile_no" : string,  
  "password": string,  
}
```

Responses

1. 500:

```
{  
  status:0,  
  message:"user has been already Registered with this Email I  
}
```

2. 200:

```
{
  status:1,
  message:"User has been Registered Successfully !"
}
```

▼ /login POST

Parameters:

body:

```
{
  "email": string,
  "password": string
}
```

Responses

1. 500:

```
{
  status: 0,
  message: "Incorrect Password"
}
```

2. 200:

```
{
  status:1,
  message:"user logged in successfully !",
  token: userToken
}
```

3. 501:

```
{
  status: 0,
```

```
message: "user not exist with this email address"
}
```

▼ /addBook POST

Parameters:

headers:

```
{
  "Authorization": userToken(string)
}
```

body:

```
{
  "bookcode" : string,
  "title" : string,
  "author" : string
  "publication" : string
  "ISBN" : Int
  "price" : Integer
}
```

Responses

1. 500:

```
{
  status: 0,
  message: "Book is already added",
  data: error
}
```

2. 200:

```
{
  status: 1,
```

```
message: "Book is added successfully",
data: user.balance
}
```

▼ /removeBook DELETE

Parameters:

headers:

```
{
  "Authorization": userToken(string)
}
```

body:

```
{
  "bookcode":String
}
```

Responses

1. 500:

```
{
  status: 0,
  message: "Error occured"
}
```

2. 200:

```
{
  status: 1,
  message: "Book has been removed successfully",
  data: [FLOAT]
}
```


▼ /updateBook POST

Parameters:

headers:

```
{  
  "Authorization": userToken(string)  
}
```

body:

```
{  
  "bookcode" : string,  
  "title" : string,  
  "author" : string  
  "publication" : string  
  "price" : Integer  
  
}
```

Responses

1. 500:

```
{  
  status:0,  
  message:"Transaction Failed"  
}
```

2. 200:

```
{  
  status:1,  
  message:"Book information has been updated Successfully"  
}
```

3. 501:

```
{
  status:0,
  message:"Unable to update Sender Balance",
  error: error
}
```

▼ /addStudent POST

Parameters:

headers:

```
{
  "Authorization": userToken(string)
}
```

body:

```
{
  "name" : string,
  "branch" : string,
  "rollno" : integer
  "email" : string
  "mobilenno" : string
  "prn" : Integer
}
```

Responses

1. 500:

```
{
  status:0,
  message: "Student is already Added.",
}
```

2. 200:

```
{
  message: "Student has been added successfully.",
  data: allTransactions,
}
```

▼ /deleteStudent DELETE

Parameters:

headers:

```
{
  "Authorization": userToken(string)
}
```

body:

```
{
  "PRN" : Integer
}
```

Responses

1. 500:

```
{
  status: 0,
  message: "Error Occured "
}
```

2. 200:

```
{
  status: 1,
  message: "Student deleted Successfully"
}
```

▼ /displayStudent GET

Parameters:

headers:

```
{  
  "Authorization": userToken(string)  
}
```

Responses

1. 500:

```
{  
  status: 0,  
  message: "Error occurred"  
}
```

2. 200:

```
{  
  status: 1,  
  message: "List of all students"  
  data: result  
}
```

▼ /updateStudent POST

Parameters:

headers:

```
{  
  "Authorization": userToken(string)  
}
```

body:

```
{
```

```
"name" : string,  
"branch" : string,  
"rollno" : integer  
"email" : string  
"mobilenno" : string  
"prn" : Integer  
}
```

Responses

1. 500:

```
{  
  status: 0,  
  message: "Error Occured "  
}
```

2. 200:

```
{  
  status: 1,  
  message: "Student Information Updated successfully"  
}
```

▼ /issueBook POST

Parameters:

headers:

```
{  
  "Authorization": userToken(string)  
}
```

Body:

```
{
```

```
"student_id": Integer
"book_id": Integer
"bookcode": Integer
}
```

Responses

1. 501:

```
{
  status: 0,
  message: "Book is not available"
}
```

2. 200:

```
{
  status: 1,
  message: "Book has been successfully Issued"
}
```

▼ /returnBook POST

Parameters:

headers:

```
{
  "Authorization": userToken(string)
}
```

body:

```
{
  "student_id": Integer,
  "book_id": Integer
  "bookcode": Integer
}
```

Responses

1. 500:

```
{
  status: 0,
  message: "This book has been already returned"
}
```

2. 200:

```
{
  status: 1,
  message: "Book has been Returned successfully with Penalty"
}
```

3. 201

If Book student returned book after expected returned date.

```
{
  status: 1,
  message: "Book has been Returned successfully with 0.0 Per"
}
```

▼ /issuedBook **POST**

Parameters:

headers:

```
{
  "Authorization": userToken(string)
}
```

body:

```
{
  "student_id": Integer
}
```

Responses

1. 500:

```
{
  status: 0,
  message: "No book is issued"
}
```

2. 200:

```
{
  status: 1,
  message: "List of all issued books"
  data:result
}
```

▼ /lostBook POST

Parameters:

headers:

```
{
  "Authorization": userToken(string)
}
```

body:

```
{
  "student_id":Integer
  "book_id": Integer
  "bookcode": Integer
}
```

Responses

1. 500:


```
{
  status: 0,
  message: "Book is not available in queue"
}
```

2. 200:

If Book student informed about lost book before expected returned date.

```
{
  status: 1,
  message: "Book has been returned successfully with ${bookF
}
```

3. 201

If Book student informed about lost book After expected returned date.

```
{
  status: 1,
  message: "Book has been returned successfully with ${bookF
}
```

▼ /Middleware-checkToken()

Responses

1. 400:

```
{
  status:0,
  message: "Please provide authorization token"
}
```

2. 401:

```
{
  status:0,
  message: "Token is not valid",
  data: error
}
```

